13. Bölüm

Sınıflar Arası İlişkiler ve SOLID İlkeleri

13.1 Unified Modeling Language

UML(Unified Modeling Language), nesneye dayalı yazılım geliştirme ile birlikte gelişen bir modelleme aracıdır. Bu nedenle 11 başlığında açıklanan nesne yönelimli birçok kavram, izleyen sayfalarda daha da pekişecektir. Adından da anlaşılacağı üzere nesneye dayalı problem çözmenin kalbi, model oluşturmaktır.

Model, gerçek dünyadaki karmaşık problemin esaslarını soyut olarak önümüze koyar. Buradan hareketle UML dilini, basit bir dil, gösterim ya da söz dizimi gibi algılamalıyız. UML dilinin, yazılımı nasıl geliştireceğimizi kesin olarak belirtmediği göz önüne alınmalıdır.

UML, bir grafik modelleme dili olup yazılım sistemlerini oluşturan ana elemanları (Ki bunlar İngilizce "artifact" olarak adlandırılıp ve el yapımı anlamına gelmektedir.) çeşitli şekillerden oluşacak şekilde tanımlamak için oluşturulmuş bir kurallar bütünüdür. Örnek verecek olursak, bir mimari projeye ilişkin maket ne ise yazılım için UML de aynı şeyi ifade eder. Yani UML, söz konusu yazılımın neyi yapacağını anlatan çeşitli şekillerin anlamlı bir şekilde bir araya getirilmesini sağlayan bir standarttır.

Yazılım mühendisliğiyle (software engineering) birlikte 1989 ve 1994 yılları arasında elliden fazla modelleme dili kullanılmaya başlanmıştır. Bu nedenle bu dönem yöntem savaşlarının yaşandığı bir dönem olarak adlandırılır. Bu yöntemlerin çoğu kendi söz dizimini oluşturmuş olmakla birlikte kullandıkları dil elemanları açısından birbirleriyle benzerlik göstermektedirler. Doksanlı yılların ortalarında üç yöntem, daha güçlü ve yaygın hale gelmiştir. Bu yöntemlerin her biri, diğerlerinin içerdiği elemanları da içeriyordu ancak her birinin diğerine karşı çeşitli güçlü özellikleri bulunuyordu. Bu yöntemler:

- Booch yönteminin tasarım (design) ve kodlama (implementation) yönleri oldukça iyi idi. *Grady Booch*, Ada diliyle oldukça çalışmıştı ve nesne yönelimli tekniklerin Ada diline uygulanması konusunda yaptığı çalışmalarla bu konuda önemli bir oyuncu olmuştur. Booch yöntemi güçlü olmasına rağmen model, birçok bulut şeklinden oluşmasından dolayı gösterim şekli oldukça sevimsizdi.
- OMT(Object Modeling Technique) tekniği, *Jim Rumbaugh* tarafından geliştirilmiş olup, analiz ve veri merkezli bilgi sistemlerinin modellenmesinde oldukça iyi bir yöntemdir.
- OOSE (Object Oriented Software Engineering) modeli **use-case** olarak bilinen bir model olup *Ivar Jacobson* tarafından geliştirilmiştir. Use-case modeli, yazılımı yapılacak sistemin davranışlarını anlamaya yönelik gelişmiş güçlü bir tekniktir.

1994 yılında *Jim Rumbaugh* ve General Electric firmasından ayrılan *Grady Booch* birlikte Rational şirketini kurarak bir araya geldiler. Bu birlikteliğin amacı, kendi yöntemlerini bir araya getirecek yeni ve tek bir yöntem olan "Unified Method" oluşturmak idi.

1995 yılında *Ivar Jacobson* da Rational şirketine katıldı. *Ivar Jacobson*'ın use-case'ler konusundaki fikirleriyle birlikte Rational şirketi, bugün UML olarak adlandırılan, yeni bir "Unified Method" geliştirdi. Üç kişiden oluşan bu grup "Three Amigos" olarak bilinir.

Yöntem savaşlarının ardından güçlü yazılım üreticileri bir araya gelerek OMG (Object Management Group) adında bir şirketler birliği kurdular[1]. Bu şirketler birliği 1997 yılında UML'e sahip çıkarak herkes tarafından kullanılan bir dil olmasını ve bir standart haline gelmesini sağlamıştır.

UML sadece, gerçek dünyadaki süreçlerin modelini ortaya koyan bir gösterim (notation) sistemidir. Süreçlerin standartlaştırılmasına yönelik bir dil değildir. UML ile ortaya çıkan modeller, süreçleri soyut (abstract) olarak tanımlayacaktır, ancak süreçlerin doğruluğu hakkında bir bilgi vermeyecektir. Yani, A şirketi için çalışan bir süreç, B şirketine uygulandığında felaketle sonuçlanabilir. Model, yukarıda da anlatıldığı üzere çözülecek probleme ilişkin bir soyutlamadır (abstraction). Etki alanı (domain) ise problemin yaşandığı gerçek dünyayı ifade eder.

Model, birbirlerine ileti (message) gönderen nesnelerden (object) oluşur. Bu modelde nesneler canlı (alive) olarak düşünülmelidir. Nesneler bir şeyler bilir (know) ve bildikleriyle bir şeyler yaparlar (do). Modelde nesneler, nesne tanımında verilen özellik (property) ve alanları (field) gösteren niteliklere (attribute) sahiptirler. Nesnelerin gösterdikleri davranış (behavior) ya da geliştirdiği yöntemler (method) ise işlem (operation) olarak adlandırılır. Nesnenin özelliklerine ait o anki değerler ise nesnenin durumunu (state) belirler. UML, sekiz tür

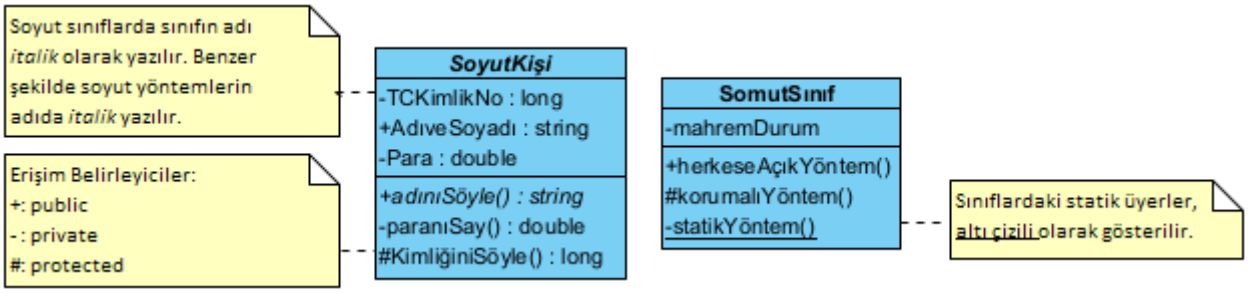
[1] OMG (Object Management Group), www.omg.org

diyagrama sahiptir ve bu kitapta **sınıf diyagramlarına** (class diagram) yer verilecektir[2].

13.2 Sınıf Diyagramları

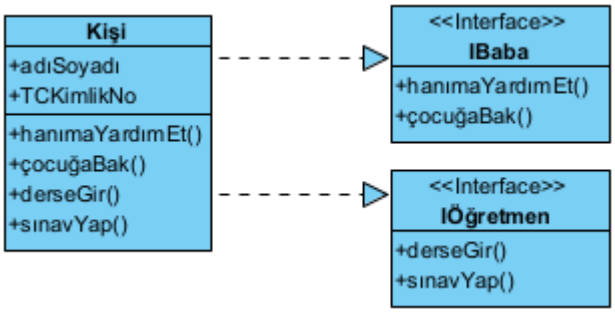
Sınıf diyagramları (class diagram), yazılımı geliştirilecek olan sisteme ait sınıfları (class) ve bu sınıflar arasındaki ilişkiyi (relationship) gösterir. Sınıf diyagramları durağandır (static). Yani sadece sınıfların birbiriyle etkileşimini (what interacts) gösterir, bu etkileşimler sonucunda ne olduğunu (what happens) göstermez.

Sınıf diyagramlarında sınıflar, bir dikdörtgen ile temsil edilir. Bu dikdörtgen üç parçaya ayrılır. En üstteki birincisine sınıf ismi yazılır. Ortadaki ikincisinde nitelikler (attribute), en alttaki ve sonuncusunda ise ve işlemler (operation) yer alır.



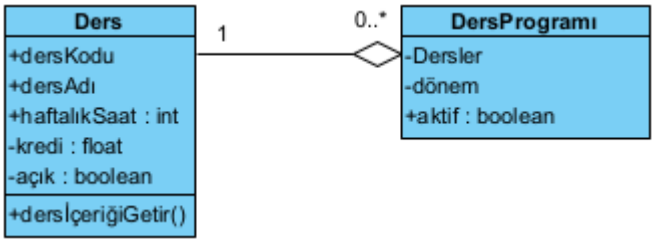
Şekil 13.1: Örnek Bir Sınıf Diyagramı

Sınıf isminin italik yazılması, sınıfın soyut (abstract class) sınıf olduğunu gösterir. Benzer şekilde işlemlerin yer aldığı kısımda, yöntemlerin italik yazılması halinde, ilgili yöntemin soyut yöntem (abstract method) olduğunu gösterir. Sınıflardaki erişim belirleyicilerin(access modifier), özel karakterlerle birbirinden ayrılır. Burada (-) karakteri mahrem (private), (+) karakteri umuma açık (public), (#) karakteri ise korumalı (protected) olduğunu anlatmaktadır. Statik (static) üyeler altı çizili olarak gösterilir. **Ara yüz gerçekleştirme** (in-



Şekil 13.2: Sınıf Diyagramlarında Arayüz Gerçekleştirme

terface realization) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen ara yüzü gösterir. Çemberler, ara yüzleri (interface) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar **lolipop diyagramı** olarak da adlandırılır. Çokluk (multiplicity), ilişkide bir sınıfın örnek (instance) sayısını gösterir. İlişkiyi gösteren



Şekil 13.3: Sınıf Diyagramlarında Sınırlama ve Çokluk

çizginin sonunda bir numara olarak gösterilir. Bu numara, mümkün olabilecek örnek sayısıdır. Çokluk bir sonraki başlıkta verilen, **iş birliği** (association), **bileşim** (composition) ve **bütünleşmeye** (aggregation) uygulanır. Çokluk örnekleri:

- `0..1`: Sıfır ya da 1 örnek.

[2] www.togethersoft.com/services/practical_guides/umlonlinecourse/index.html

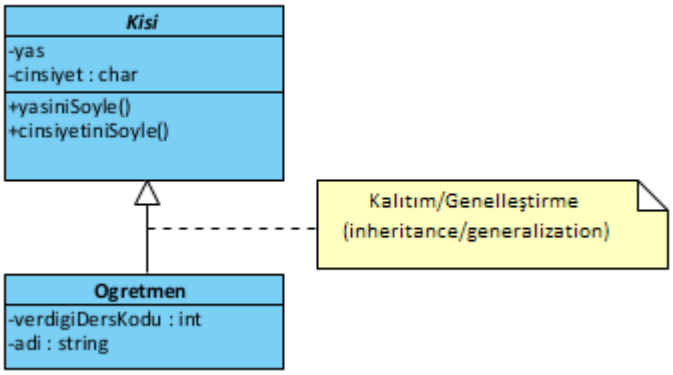
- `0..` veya `*`: Örnek sayısı sınırsızdır.
- `1`: Yalnızca 1 örnek
- `1..`: En az bir örnek

13.3 Sınıflar Arası İlişkiler

Nesneleri imal ettiğimiz sınıflar arasında ya ilişki yoktur ya da aşağıda sıralanan altı çeşit ilişki vardır.

§13.3.1 Genelleştirme İlişkisi

Genelleştirme (generalization), taban sınıfla (base class) ile türemiş sınıf (derived class) arasındaki **kalıtım** (inheritance) ilişkisidir. Örneği 11.9 başlığında verilmiştir. Aşağıda `Kisi` sınıfı ile bu sınıftan miras alan `Ogretmen` sınıfının UML diyagramı verilmiştir.

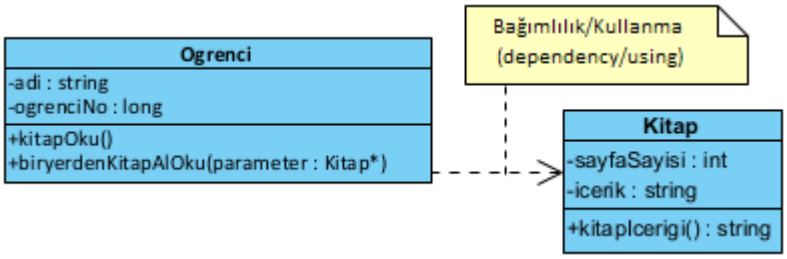


Şekil 13.4: Genelleştirme İlişkisi Örneği

§13.3.2 Bağımlılık İlişkisi

Bağımlılık (dependency) ya da bir başka deyişle **kullanma** (using), bir sınıf ile diğer arasındaki geçici ilişkidir. Burada kullanan sınıf istemci (client) diğer sınıf ise sağlayıcı (supplier) olarak adlandırılır. İki türlü bağımlılık vardır;

1. İstemci sınıf, sağlayıcıyı sınıfı **parametre olarak** kullanır (dependency as a parameter).
2. İstemci sınıf, sağlayıcı sınıftan **örnekleme** yapar (dependency as an instantiation).



Şekil 13.5: Bağımlılık İlişkisi Örneği

Aşağıda `Ogrenci`-`Kitap` ilişkisini gösteren bir bağımlılık örneği verilmiştir.

```
#include <iostream>
#include <string>
class Kitap {
public:
    std::string kitapIcerigi() {
        return icerik;
    }
    Kitap(int pSayfaSayisi, std::string pIcerik) : sayfaSayisi(pSayfaSayisi), icerik(pIcerik) {}
private:
    std::string icerik;
    int sayfaSayisi;
};
class Ogrenci {
public:
    void kitapOku() {  //Dependency as Instantiation
```

```
        // Yöntem içinde oluşturma (Sıkı bağlılık örneği)
        Kitap* kitap = new Kitap(150, "C++ ile Nesne Yonelimli Programlama Notlari");
        std::cout << "kitap imal edilip kullaniliyor..." << std::endl;
        std::string notlar = kitap->kitapIcerigi();
        delete kitap; // Yöntem bitmeden temizlik yapılıyor
    }
    void biryerdenKitapAlOku(Kitap* pKitap) { //Dependency as a Parameter
        // Dışarıdan enjekte edilme (Esnek bağımlılık örneği)
        if(pKitap) {
            std::cout << "kitap disardan alinip kullaniliyor..." << std::endl;
            std::string okunacak = pKitap->kitapIcerigi();
        }
    };
    Ogrenci(std::string pAdi, int pNo) : adi(pAdi), no(pNo) {}
private:
    std::string adi;
    int no;
};
int main() {
    Ogrenci* ogrenci = new Ogrenci("Ali", 12000);
    ogrenci->kitapOku();
    Kitap* kitap = new Kitap(350, "Kurumsal yazilim Gelistirme");
    ogrenci->biryerdenKitapAlOku(kitap);
    delete kitap;
    delete ogrenci;
}
```

Bu program, nesne yönelimli programlamada sınıflar arasındaki **bağımlılık** (dependency) ilişkisini ve bu bağımlılığın iki farklı şekilde nasıl kurulabileceğini gösteren çok temiz bir örnektir. Programda `Ogrenci` sınıfı, işini yapabilmek (kitap okumak) için `Kitap` sınıfına ihtiyaç duyar. Örnek Program, bir nesnenin başka bir nesneye olan bağımlılığını iki ana senaryoda ele almıştır;

1. **Örnekleme Olarak Bağımlılık (Dependency as Instantiation)**: `kitapOku()` yöntemi içinde, `Kitap` nesnesi metodun içinde `new` ile oluşturuluyor.
   - **Özellik**: `Ogrenci` nesnesi, hangi kitabın okunacağına kendisi karar verir ve kitabın yaşam döngüsünü (oluşturma ve silme) kendisi yönetir.
   - **Risk**: Bu durum **sıkı bağlılık** (tight coupling) yaratır. Eğer `Kitap` sınıfının yapısı (yapıcı parametreleri gibi) değişirse, `Ogrenci` sınıfının içindeki bu yöntemi da değiştirmek zorunda kalırız.
2. **Parametre Olarak Bağımlılık (Dependency as a Parameter)**: `biryerdenKitapAlOku(Kitap* pKitap)` yönteminde, kitap nesnesi dışarıdan bir parametre olarak geliyor.
   - **Özellik**: Bu, modern yazılım mimarilerinde **bağımlılık enjeksiyonu** (dependency injection) kavramının temelidir. `Ogrenci` nesnesi, kendisine hangi kitap verilirse onu okur.
   - **Avantaj**: Daha esnektir. Öğrenciye farklı kitap nesneleri (roman, ders kitabı vb.) dışarıdan kolayca gönderilebilir.

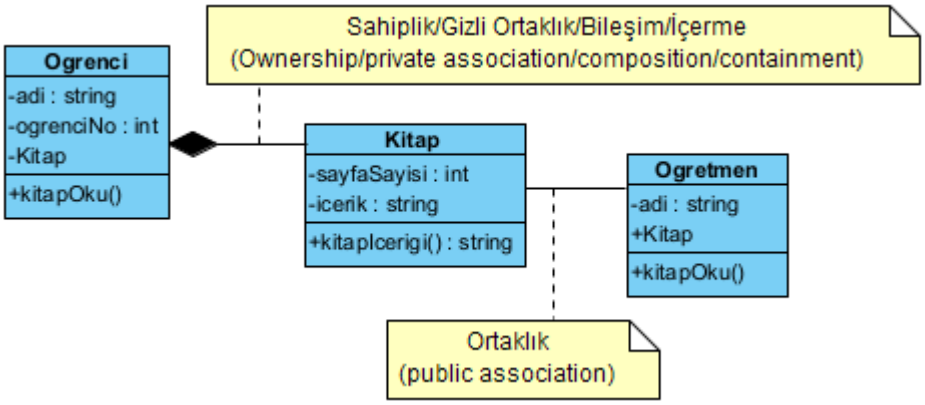
§13.3.3 Sahiplik ve Ortaklık İlişkisi

Sahiplik (ownership) ve **ortaklık** (public association) ilişkileri, iki farklı sınıf arasındaki sürekli olan bütün-parça (whole-part) ilişkileridir. Bunlar;

1. Sahiplik ilişkisinde, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. İlişkinin sürekli olabilmesi için kullanılan sınıftan bir alan değişkeni (field) mahrem (`private`) olarak tanımlanır. Bu nedenle **içerme** (containment), **gizli ortaklık** (private association) veya **bileşim** (composition) olarak da adlandırılır.
2. Ortaklık ilişkisinde, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. Ancak bu nesnenin dışarıdan değiştirilmesine izin verir. İlişkinin geçici olabilmesi için kullanılan sınıftan bir alan değişkeni (field) umuma açık (`public`) olarak tanımlanır.

Aşağıda `Kitap`, `Ogrenci` ve `Ogretmen` nesnelerinin olduğu bir ortaklık örneği verilmiştir. `Ogrenci`, `Kitap` nesnesine sahip iken `Ogretmen` nesnesi `Kitap` ile ortaklık ilişkisindedir;

```
#include <iostream>
#include <string>
class Kitap {
public:
    std::string kitapIcerigi() {
```



Şekil 13.6: Sahiplik ve Ortaklık İlişkisi Örneği

```
        return icerik;
    }
    Kitap(int pSayfaSayisi, std::string pIcerik): sayfaSayisi(pSayfaSayisi), icerik(pIcerik) { }
private:
    std::string icerik;
    int sayfaSayisi;
};
class Ogrenci { // private association to kitap
private:
    std::string adi;
    int no;
    Kitap* ptrKitap;
public:
    Ogrenci(std::string pAdi,int pNo): adi(pAdi),no(pNo) {
        //SAHİPLİK İLİŞKİSİ: kitap nesnesi ile ogrenci nesnesi birlikte imal ediliyor
        ptrKitap=new Kitap(150,"C ile Yapısal Programlama");
    }
    void kitapOku() {
        std::cout << "Öğrenci ile birlikte imal edilen kitap kullanılıyor..." << std::endl;
        std::cout << "Okunan Kitap:" << ptrKitap->kitapIcerigi() << std::endl;
        //...
    }
    // YIKICI: Bellek sızıntısını önler. Çünkü sınıf içinde Kitap* ptrKitap; talimatı var.
    ~Ogrenci() { delete ptrKitap; }
};
class Ogretmen { // Kitapsız öğretmen olmaz! Ama kitabını değiştirebilir.
private:
    std::string adi;
    Kitap* ptrKitap;  /* ORTAKLIK İLİŞKİSİ: public association to Kitap, setKitap() ve getKitap()
     ↪ yöntemleri ile Özellik haline getirilip umuma açık hal getiriliyor! */
public:
    Ogretmen(std::string pAdi): adi(pAdi) {
        //kitap nesnesinin referansı ile ogretmen nesnesi birlikte imal ediliyor:
        ptrKitap=new Kitap(200,"C++ ile Nesne Yönelimli Programlama");
    }
    // Kitap* üyesi get ve set yöntemleriye public hale getiriliyor.
    void setKitap(Kitap* pKitap) {
        if (ptrKitap!=nullptr) delete ptrKitap;
        ptrKitap=pKitap;
    }
    Kitap* getKitap() {
        return ptrKitap;
    }
    void kitapOku() {
        std::cout << "Öğretmen ile birlikte imal edilen kitap kullanılıyor..." << std::endl;
        std::cout << "Okunan Kitap:" << ptrKitap->kitapIcerigi() << std::endl;
```

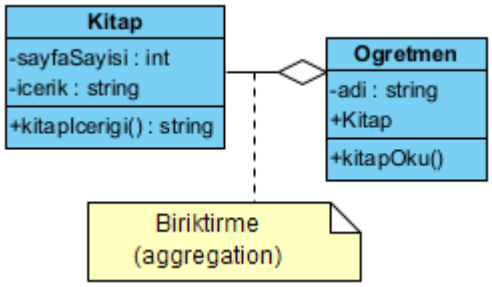
```
        //...
    }
};
int main() {
    Ogrenci* ogrenci=new Ogrenci("Ali",12000);
    ogrenci->kitapOku();
    Ogretmen* ogretmen=new Ogretmen("Ilhan");
    ogretmen->kitapOku();
    Kitap* ogretmeninOkuduguKitap=ogretmen->getKitap();
    std::cout << "Orgetmenin Okuduğu Kitaba Main İçinden Ulaşılabiliyor;" << std::endl
    << "Okunan Kitap:" << ogretmeninOkuduguKitap->kitapIcerigi() << std::endl;
    Kitap* baskaKitap=new Kitap(250,"Kurumsal Yazılım Geliştirme");
    ogretmen->setKitap(baskaKitap);
    ogretmen->kitapOku();
    delete ogrenci;
    delete ogretmen;
    delete baskaKitap;
}
```

Örnek program, nesne yönelimli programlamada nesneler arasındaki sahiplik (ownershipownership) ve birliktelik (association) ilişkilerini, özellikle de mahrem (private) ve umuma açık (public) erişim düzeylerinin bu ilişkileri nasıl kısıtladığını veya esnettiğini gösterir. Programdaki temel kavramları ve teknik detayları;

1. **Sahiplik** (ownership/ composition/ private association): `Ogrenci`-`Kitap` Sınıfları
   - **Durum**: `Ogrenci` nesnesi yaratıldığında `Kitap` nesnesi de onun içinde otomatik olarak `new` ile oluşturuluyor.
   - **Gizlilik**: `ptrKitap` üyesi `private` olduğu ve dışarıya açan bir `get` metodu bulunmadığı için, `main()` fonksiyonu içinden öğrencinin kitabına erişilemez veya bu kitap değiştirilemez.
   - **Yaşam Döngüsü**: Kitap, öğrenciye bağımlıdır.
2. **Ortaklık** (public association): `Ogretmen`-`Kitap` Sınıfları
   - **Durum**: Öğretmen de bir kitapla doğar, ancak `getKitap()` ve `setKitap()` yöntemleri sayesinde bu özellik (property) dış dünyaya açılmıştır.
   - **Esneklik**: `main()` fonksiyonu içinde öğretmenin okuduğu kitaba ulaşabildik (`getKitap()`) ve hatta kitabını başka bir kitap nesnesiyle değiştirebildik (`setKitap()`).
   - **Bağımsızlık**: Öğretmen nesnesi hala hayattayken, ona dışarıdan bambaşka bir kitap nesnesi **enjekte edilebilir**.

§13.3.4 Biriktirme İlişkisi

Biriktirme (aggregation) ilişkisi, iki farklı sınıf arasındaki sürekli olan bütün-parça (whole-part) ilişkisidir. Ancak **sahiplik** (ownership) ve **ortaklık** (public association) ilişkilerinden farklı olarak burada nesnelerin yaşam döngüleri birbirinden ayrılmıştır.



Şekil 13.7: Biriktirme İlişkisi Örneği

```
#include <iostream>
#include <string>
class Kitap {
public:
    std::string kitapIcerigi() { return icerik; }
    Kitap(int pSayfaSayisi, std::string pIcerik) : sayfaSayisi(pSayfaSayisi), icerik(pIcerik) {}
private:
    std::string icerik;
    int sayfaSayisi;
```

```
};
class Ogretmen {
private:
    std::string adi;
    Kitap* ptrKitap; /* setKitap() ve getKitap() yöntemleri ile Özellik haline getirilip umuma
     ↪ açık hal getiriliyor! */
public:
    /* BİRİKTİRME İLİŞKİSİ: Yapıcı yöntemde varolan bir kitabın adresi alınarak Ogretmen imal
     ↪ ediliyor. Kitap olmadan asla imal edilemez! */
    Ogretmen(std::string pAdi, Kitap* ppKitap) : adi(pAdi), ptrKitap(ppKitap) {}
    void setKitap(Kitap* pKitap) { ptrKitap = pKitap; }
    Kitap* getKitap() { return ptrKitap; }
    void kitapOku() {
        if (ptrKitap) { // Güvenlik kontrolü
            std::cout << adi << " su kitabi okuyor: " << ptrKitap->kitapIcerigi() << std::endl;
        }
    }
};
int main() {
    // 1. Parça (Kitap) önce oluşturulur
    Kitap kitap1(150, "C++ ile Nesne Yonelimli Programlama");
    Kitap kitap2(200, "Veri Yapilari ve Algoritmalar");
    // 2. Bütün (Ogretmen) parçaya bağlanarak oluşturulur
    Ogretmen ilhan("Ilhan", &kitap1);
    ilhan.kitapOku();
    // 3. Parça çalışma zamanında değiştirilebilir
    ilhan.setKitap(&kitap2);
    std::cout << "Ogretmen kitabini degistirdi." << std::endl;
    ilhan.kitapOku();
}
```

Programdaki biriktirme (aggregation) ilişkisi `Ogretmen` ve `Kitap` arasında zayıf bir sahiplik ilişkisidir;

- **Bağımsız Yaşam Döngüsü**: `Kitap` nesnesi, `Ogretmen` nesnesinden önce `main()` içinde oluşturulur. Burada `Öğretmen` emekli olsa (nesne silinse) bile kitap nesnesi `main()` içinde var olmaya devam eder.
- **Zorunlu Varlık**: `Ogretmen` yapıcısı olan `Ogretmen(..., Kitap* ppKitap)` bir kitap adresi beklediği için, sistemde bir kitap olmadan öğretmen nesnesi imal edilemez.
- **Esneklik**: `setKitap()` yöntemi sayesinde `Ogretmen` elindeki kitabı bırakıp başka bir kitabı alabilir.

13.4 Sınıf Tasarlama İlkeleri

Nesne yönelimli programlama (object oriented programming) kodumuzda değişim yönetimi (change management), gösterici kullanımı yerine referansları kullanarak güvenli kod (safe code) aynı kodları tekrar yazmak (duplicate code) yerine kodların yeniden kullanımı (reusing) sağlayan aşağıdaki özellikleri kullanırız;

- Kod küçüldüğü için kalıtım (inheritance),
- Veri ve kontrol soyutlaması (abstraction) yaparak daha güvenli bileşen tasarladığımız için sarmalama (encapsulation),
- Yazılım mimarisinin daha esnek ve genişleyebilir olması için çok biçimlilik (polymorphism).

Ayrıca yazılım geliştirmede;

- Zayıf bağlaşımın (loose couple) sağlanması için yazılımın bir noktasında olabilecek değişikliğin, diğer kısımlarda değişiklik gerektirmeyecek şekilde tasarlanması gerekir. Yani yapılan değişiklik, çorap söküğü gibi yazılımın diğer kısımlarını değiştirmemelidir.
- Aynı çağrışım kümesine ilişkin bütün bileşenler bir arada geliştirilmeli diğer kümelerle karıştırılmamalıdır (seperation of concern). Yani bileşenler arasında yüksek uyum (high cohesion) olmalıdır.

İşte yeniden kullanım, zayıf bağlaşım ve yüksek uyum sağlamak için programcı, nesne yönelimli programlama özellikleriyle birlikte temel ilkelere (principle) uyar. Programcı kodu tasarlarken, yazarken ve yeniden düzenlerken bu ilkeleri aklında tutarsa; Kod çok daha temiz, genişletilebilir ve test edilebilir olur. İlkelerin İngilizce baş harflerinin alınarak **SOLID ilkeleri** (SOLID principles) olarak isimlendirilir.

§13.4.1 Tek Sorumluluk İlkesi

Bu ilke ile SRP (Single Responsibility Principle) sınıflar, tasarlanırken birden fazla işten sorumlu tutulmamalıdır. Mümkün olduğunca bir işle ilgili olarak tasarlanmalıdır. Bir sınıfın tek bir şey yapması gerektiğini ve

dolayısıyla değişmesi için tek bir nedeninin olması gerektiğini belirtir.

```
// Kötü Örnek: Birden fazla sorumluluk
class Employee {
    void calculatePay() { /* ... */ }
    void saveToDatabase() { /* ... */ }
    void generateReport() { /* ... */ }
};

// İyi Örnek: Her sınıf tek sorumluluk
class Employee {
    // Sadece employee verisi
};
class PayrollCalculator {
    void calculate(const Employee& emp) { /* ... */ }
};
class EmployeeRepository {
    void save(const Employee& emp) { /* ... */ }
};
class ReportGenerator {
    void generate(const Employee& emp) { /* ... */ }
};
```

§13.4.2 Açık-Kapalı İlkesi

Bu ilke ile OCP (Open Closed Principle) sınıflar, **değişikliklere kapalı** (closed for modification) **eklentilere açık** (open for extension) şekilde tasarlanmalıdır. Bu şekilde değişikliklere açık genişleyebilir modüllere sahip oluruz. Değişikliklere kapalı olmak, mevcut bir sınıfın kodunu değiştirmemek anlamına gelirken, eklentilere açık olma yeni işlevler eklemek anlamına gelir.

```
// Eklentiler için açık, değişiklik için kapalı
class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
};
class Circle : public Shape {
private:
    double radius_;
public:
    double area() const override {
        return M_PI * radius_ * radius_;
    }
};
class AreaCalculator {
public:
    //std::vector ve std::unique_ptr için şablonlar başlığını inceleyiniz
    double totalArea(const std::vector<std::unique_ptr<Shape>>& shapes) {
        double total = 0;
        for (const auto& shape : shapes) {
            total += shape->area();
        }
        return total;
    }
};
```

§13.4.3 Liskov İkame İlkesi

Bu ilkede LSP (Liskov Substitution Principle) alt sınıflar türediği sınıfların yerine konulabilmelidir. Bu prensipten ilk olarak *Barbar Liskov* tarafından bahsedilmiştir. Alt sınıflar, türediği sınıf davranışlarıyla kullanılabilmelidir. Burada amaç, alt sınıfların daha çok değişebileceği göz önüne alınarak değişimlerden an az şekilde etkilenmektir.

```
// Alt sınıflar üst sınıf yerine kullanılabilmeli
class Bird {
```

```
public:
    virtual ~Bird() = default;
    virtual void eat() { /* ... */ }
};
class FlyingBird : public Bird {
public:
    virtual void fly() { /* ... */ }
};
class Sparrow : public FlyingBird {
    void fly() override { /* Can fly */ }
};
class Penguin : public Bird {
    // Uçamaz, FlyingBird'den türemez
};
```

Türemiş sınıfından bir nesneyi, Taban sınıfından bir nesne bekleyen herhangi bir yönteme parametre olarak geçirdiğimizde yöntemin herhangi bir tuhaf çıktı vermemesi gerektiği anlamına gelir. Bu beklenen davranıştır, çünkü kalıtım kullandığımızda alt sınıfın, üst sınıfın sahip olduğu her şeyi miras aldığını varsayarız. Alt sınıf davranışı genişletir ancak asla daraltmaz. Dolayısıyla bir sınıf bu ilkeye uymadığında, tespit edilmesi zor bazı kötü hatalara yol açar.

§13.4.4 Ara Yüzleri Ayırma İlkesi

Bu ilke ile ISP (Interface Segregation Principle) genel amaçlı ara yüzler yerine istemciye bağlı ara yüzler tasarlanmalıdır. Yani roller, birbirinden ayrı olacak şekilde tanımlanmalıdır. Birden fazla rol tek bir rol altında olacak şekilde düşünülmemelidir. Bu nedenle buna rollerin ayrılması prensibi RSP (Role Segregation Principle) olarak da adlandırılır.

```
// Küçük, işe özel (spesific) interface'ler
class IPrintable {
public:
    virtual void print() = 0;
};
class IScannable {
public:
    virtual void scan() = 0;
};
class IFaxable {
public:
    virtual void fax() = 0;
};
class SimplePrinter : public IPrintable {
    void print() override { /* ... */ }
};
class MultiFunctionPrinter : public IPrintable, public IScannable, public IFaxable {
    void print() override { /* ... */ }
    void scan() override { /* ... */ }
    void fax() override { /* ... */ }
};
```

§13.4.5 Bağımlılıkları Ters Çevirme İlkesi

Bu ilke DIP (Dependency Inversion Principle), sınıflarımızın somut sınıflara ve fonksiyonlara değil, ara yüzlere veya soyut sınıflara bağlı olması gerektiğini belirtir. Nesne yönelimli programlamanın en önemli ilkesidir. Aslında bu ilke ile açık-kapalı ilkesi doğrudan birbiriyle ilişkilidir.

```
// Soyutlamalara bağımlı ol, somut sınıflara değil
class IDatabase {
public:
    virtual ~IDatabase() = default;
    virtual void save(const std::string& data) = 0;
};
class MySQLDatabase : public IDatabase {
    void save(const std::string& data) override {
        // MySQL implementation
```

```
    }
};
class PostgreSQLDatabase : public IDatabase {
    void save(const std::string& data) override {
        // PostgreSQL implementation
    }
};
class UserService {
private:
    // std::unique_ptr için şablonlar başlığını inceleyiniz
    std::unique_ptr<IDatabase> database_;
public:
    UserService(std::unique_ptr<IDatabase> db):database_(std::move(db)) {}
    void saveUser(const std::string& user) {
        database_->save(user);
    }
};
```

13.5 Düşün ve Çöz

- **Düşün**: UML sınıf diyagramında birliktelik (composition) ile biriktirme (aggregation) arasındaki farkı somut bir örnekle gösterin. 'Araba–Motor' ilişkisi ile 'Üniversite–Öğrenci' ilişkisi bu iki kavramdan hangisine karşılık gelir?
- **Çöz**: SOLID ilkelerinden 'Tek Sorumluluk İlkesi'ni ihlal eden bir sınıf örneği yazın, ardından bu sınıfı ilkeye uygun hale getirin. Refactoring öncesi ve sonrası kodları karşılaştırın.
- **Düşün**: Açık-Kapalı İlkesi'ne göre bir sınıf 'genişlemeye açık, değişikliğe kapalı' olmalıdır. Bir ödeme sistemi örneği üzerinden bu ilkeyi uygulamak için kalıtım veya kompozisyon kullanarak nasıl tasarım yaparsınız?
- **Çöz**: Liskov İkame İlkesi'ni ihlal eden klasik 'Kare–Dikdörtgen' örneğini açıklayın. Bu ihlalin programın davranışını nasıl bozduğunu kodla gösterin ve çözümünü önerin.
- **Çöz**: Bağımlılıkları Ters Çevirme İlkesi neden yüksek seviyeli modüllerin düşük seviyeli modüllere bağımlı olmaması gerektiğini söyler? Bağımlılık enjeksiyonu (dependency injection) bu ilkenin pratikte nasıl uygulandığını örnekleyin.
- **Düşün**: Sıkı bağlaşım (tight coupling) ile gevşek bağlaşım (loose coupling) arasındaki farkı UML diyagramı çizerek gösterin. Gevşek bağlaşım sağlamak için hangi tasarım teknikleri kullanılabilir?